

UI Colors

PHASE 1: Introduction to UI Colors

VERBATIM STREAM:

- [spoken_transcript] Colors are one of those things where the more you learn, the more you realize how much you didn't know before. And it gets complicated real fast. But for UI stuff, you don't need to become a color scientist. In my experience, colors are honestly the easiest part of the UI. The tricky bit is knowing when to stop playing and just pick something that works.
- [spoken_transcript] Here's what actually works. Neutral colors for background, text, borders, and a lot of other elements. The brand or primary color for main actions and to add a bit of character. And finally, maybe some semantic colors to communicate different states. These are all the colors you'll ever need to design apps and UIs.

TEACHABLES:

1. TYPE: rule

VERBATIM: Here's what actually works. Neutral colors for background, text, borders, and a lot of other elements. The brand or primary color for main actions and to add a bit of character. And finally, maybe some semantic colors to communicate different states.

PHASE 2: Color Formats and Shades

VERBATIM STREAM:

- [spoken_transcript] When I say colors, what I actually mean is shades. You will see this everywhere on a UI. Like when you hover over a button or a gradient background. To create shades, you need to pick the right color format. Most people are familiar with Hex and RGB color values, but they are the worst when it comes to creating a color palette.
- [spoken_transcript] Like these three shades of gray we have for our background colors. They look very similar visually, but the code doesn't. Now compare that with the more intuitive color format, and the code actually makes sense. You have Hue or the actual color on the color wheel that goes from 0 to 360. Then the saturation, which goes from 0 to 100, controls the intensity of that color.
- [spoken_transcript] In our case we have set it to zero, that means no matter what color you choose for the hue, it's always gonna be neutral. And finally, we use the lightness value to create a bunch of shades. Simple and easy math to create a harmonic palette without any guesswork.

TEACHABLES:

1. TYPE: definition

VERBATIM: You have Hue or the actual color on the color wheel that goes from 0 to 360. Then the saturation, which goes from 0 to 100, controls the intensity of that color.

2. TYPE: procedure

VERBATIM: In our case we have set it to zero, that means no matter what color you choose for the hue, it's always gonna be neutral. And finally, we use the lightness value to create a bunch of shades.

PHASE 3: Creating the Dark Mode Palette

VERBATIM STREAM:

- [spoken_transcript] Now that we've learned how to play with colors, let's create the most prominent colors of a UI. Background and text colors. I'll start with dark mode because let's be honest, it slaps, but yes, we will cover light mode as well.
- [spoken_transcript] Saturation can stay at zero because we are creating a neutral palette. Without saturation, hue doesn't matter anyway. I will cover hue and some other important things later in the video.
- [spoken_transcript] With hue and saturation both set to zero, we will start with 0% lightness for our base color. Then 5% for the next shade that can be used for cards and other surface elements. And finally 10% to highlight the most important or raised elements.
- [spoken_transcript] The lighter elements appear on top, and feel closer to the user. So obviously only use them for important stuff.
- [spoken_transcript] Same thing with text. Keep a sharper and high contrast color for headings and important elements, and use a slightly muted shade for the rest. Still legible, just not in your face all the time. That's why we don't use 100% lightness for the headings, it would look too harsh on the eyes. Speaking of harsh, let's flip these colors for light mode.

TEACHABLES:

1. TYPE: procedure

VERBATIM: With hue and saturation both set to zero, we will start with 0% lightness for our base color. Then 5% for the next shade that can be used for cards and other surface elements. And finally 10% to highlight the most important or raised elements.

2. TYPE: rule

VERBATIM: The lighter elements appear on top, and feel closer to the user. So obviously only use them for important stuff.

3. TYPE: procedure

VERBATIM: Keep a sharper and high contrast color for headings and important elements, and use a slightly muted shade for the rest. Still legible, just not in your face all the time. That's why we don't use 100% lightness for the headings

PHASE 4: Creating the Light Mode Palette

VERBATIM STREAM:

- [spoken_transcript] Start by subtracting the lightness value by 100. That'll give you a good starting point. Then we'll use our eyes and some common sense to adjust the values manually.
- [spoken_transcript] For example, why does the top element have the darkest color? Light usually comes from the top, so it should be the lightest, and the base color should be the darkest. That's why I name my background colors as bg-dark or bg-light. So bg-dark will be the darkest and bg-light will be the lightest shade, whether it's dark or light mode.
- [spoken_transcript] But we can't do that with text colors. So use names that make sense for both cases.

TEACHABLES:

1. TYPE: procedure
 VERBATIM: Start by subtracting the lightness value by 100. That'll give you a good starting point. Then we'll use our eyes and some common sense to adjust the values manually.
2. TYPE: rule
 VERBATIM: Light usually comes from the top, so it should be the lightest, and the base color should be the darkest.
3. TYPE: convention
 VERBATIM: That's why I name my background colors as bg-dark or bg-light. So bg-dark will be the darkest and bg-light will be the lightest shade, whether it's dark or light mode.

PHASE 5: Recap and CSS Implementation

VERBATIM STREAM:

- [spoken_transcript] So we have our background and text colors. Before we move forward, let's quickly recap how we got here. We picked the HSL color format, so the colors actually make sense in code. We kept hue and saturation at zero because we want a neutral color palette, and without any saturation, hue doesn't matter anyway.
- [spoken_transcript] Then we just played with the lightness value to create three shades for the background and two shades for text colors. And finally, we flipped these colors for light mode by subtracting the lightness value from 100. That gave us a good starting point, but bg-dark and bg-light didn't make any sense, so we switched them out to get our final colors, and that's all we've done so far.
- [spoken_transcript] All right, let's shift gears and write some CSS. First of all, set your colors like this. Your default theme goes in the root, and then the other theme can go in the body selector like this. Then, one line of JavaScript to toggle it back and forth. Or just wrap your light mode in this media query, to automatically adapt to users' preferred theme.
- [spoken_transcript] There are many ways to do this, and your implementation will depend on your framework, or how much control you want to give the user. But the main task is defining your colors, the rest can be handled by the agent.

- [spoken_transcript] For this video, I'm gonna stick with this simple logic. Define my color variables like this, and then use those variables to set the background and text colors.

TEACHABLES:

1. TYPE: procedure

VERBATIM: First of all, set your colors like this. Your default theme goes in the root, and then the other theme can go in the body selector like this. Then, one line of JavaScript to toggle it back and forth. Or just wrap your light mode in this media query, to automatically adapt to users' preferred theme.

PHASE 6: Enhancing the UI (Borders, Gradients, Highlights, Shadows)

VERBATIM STREAM:

- [spoken_transcript] At this point, our UI looks too boring. So to fix that, we are gonna use four more properties. As always, let's start with dark mode.
- [spoken_transcript] The border should be clearly visible, but not too distracting.
- [spoken_transcript] Then, create a simple gradient using our background colors. If it feels too distracting, we can tweak it a bit and reveal the full gradient on hover. This way, it looks shiny on top, like the light is coming from above.
- [spoken_transcript] To really sell this effect, we can use a lighter color for the top border, and I'm calling it, you guessed it, highlight.
- [spoken_transcript] Now let's see how this looks in light mode. Since we haven't defined our gradients for light mode yet, it's still showing the dark mode version. So let's just copy the border, gradient and highlight into light mode first.
- [spoken_transcript] This is nice, but we can make it 10 times better. First of all, the highlight in light mode doesn't look like a highlight, but a plain border. So let's bump the lightness all the way up. Now the border doesn't make any sense and looks ugly as hell.
- [spoken_transcript] To solve this issue, I'm gonna create a new variable that can be applied directly as the border. Then, we'll change the border color to match the background color in light mode, so it blends into the card.
- [spoken_transcript] And finally, let's add a nice shadow to create some depth, because where there's light, there's a shadow. Shadows need some transparency, so you'll need to use an alpha value that goes from zero to one. Always mix a darker and shorter shadow, with a lighter and longer one, to achieve a more realistic effect. But did you notice? We've come this far without even touching hue or saturation.

TEACHABLES:

1. TYPE: rule

VERBATIM: The border should be clearly visible, but not too distracting.

2. TYPE: procedure

VERBATIM: create a simple gradient using our background colors. If it feels too distracting, we can tweak it a bit and reveal the full gradient on hover.

3. TYPE: procedure
VERBATIM: To really sell this effect, we can use a lighter color for the top border, and I'm calling it, you guessed it, highlight.
4. TYPE: procedure
VERBATIM: First of all, the highlight in light mode doesn't look like a highlight, but a plain border. So let's bump the lightness all the way up.
5. TYPE: procedure
VERBATIM: I'm gonna create a new variable that can be applied directly as the border. Then, we'll change the border color to match the background color in light mode, so it blends into the card.
6. TYPE: procedure
VERBATIM: Shadows need some transparency, so you'll need to use an alpha value that goes from zero to one. Always mix a darker and shorter shadow, with a lighter and longer one, to achieve a more realistic effect.

PHASE 7: Hue and Saturation

VERBATIM STREAM:

- [spoken_transcript] So let's see what hue and saturation can do for your designs. I've made a tool that lets you do just that in real time. I've already set the right contrast, gradient, highlight and shadows. All you need to do is play with hue and saturation.
- [spoken_transcript] Want a cool and vibrant look, or something warm and neutral. This lets you try everything and find the perfect balance for your project.
- [spoken_transcript] Then just flip this switch and see if the light mode needs any tweaking. Make sure to check all the colors, especially your primary and secondary ones, since they are mainly used for buttons and hover states. Once you're happy with the results, just click this button to copy the colors.

PHASE 8: Advanced Color Formats (OKLCH)

VERBATIM STREAM:

- [spoken_transcript] What the hell is OKLCH, and why is it the default color format. Forget the OK part for a second. Just look at LCH. L and H are lightness and hue, which you already know from HSL. C stands for Chroma, which is kinda like saturation. But instead of going from 0 to 100, it goes from 0 to 0.4. And for UI work, you rarely need more than 0.15 or 0.2.
- [spoken_transcript] As I said in the beginning, colors are one of those things where the more you learn, the more you realize how much you didn't know. And it gets complicated real fast.
- [spoken_transcript] Hue is the same as before, 0 to 360. Chroma goes from 0 to 0.2, and lightness goes from 0 to 1.
- [spoken_transcript] Actually, LCH is a color format on its own. OKLCH is the newer and better version of it. With the v4 update, Tailwind has also switched to OKLCH as the default color format. So let's compare these three color formats and see how they differ.

- [spoken_transcript] With HSL, the biggest issue is how it handles lightness. The dark and light shades lose all saturation. With LCH and OKLCH, the increments look way more natural.
- [spoken_transcript] By the way, I'll show you some more variations and leave it up to you to decide which one looks better. Feel free to pause or take screenshots, since I'll be going through them quickly.
- [spoken_transcript] So what do you think? You can still just copy the HSL code if you want. The theme includes everything. Both color formats for dark and light mode. Just copy paste the code into your CSS file and start designing.

TEACHABLES:

1. TYPE: definition
 VERBATIM: L and H are lightness and hue, which you already know from HSL. C stands for Chroma, which is kinda like saturation. But instead of going from 0 to 100, it goes from 0 to 0.4. And for UI work, you rarely need more than 0.15 or 0.2.
2. TYPE: definition
 VERBATIM: Hue is the same as before, 0 to 360. Chroma goes from 0 to 0.2, and lightness goes from 0 to 1.
3. TYPE: definition
 VERBATIM: Actually, LCH is a color format on its own. OKLCH is the newer and better version of it.
4. TYPE: comparison
 VERBATIM: With HSL, the biggest issue is how it handles lightness. The dark and light shades lose all saturation. With LCH and OKLCH, the increments look way more natural.